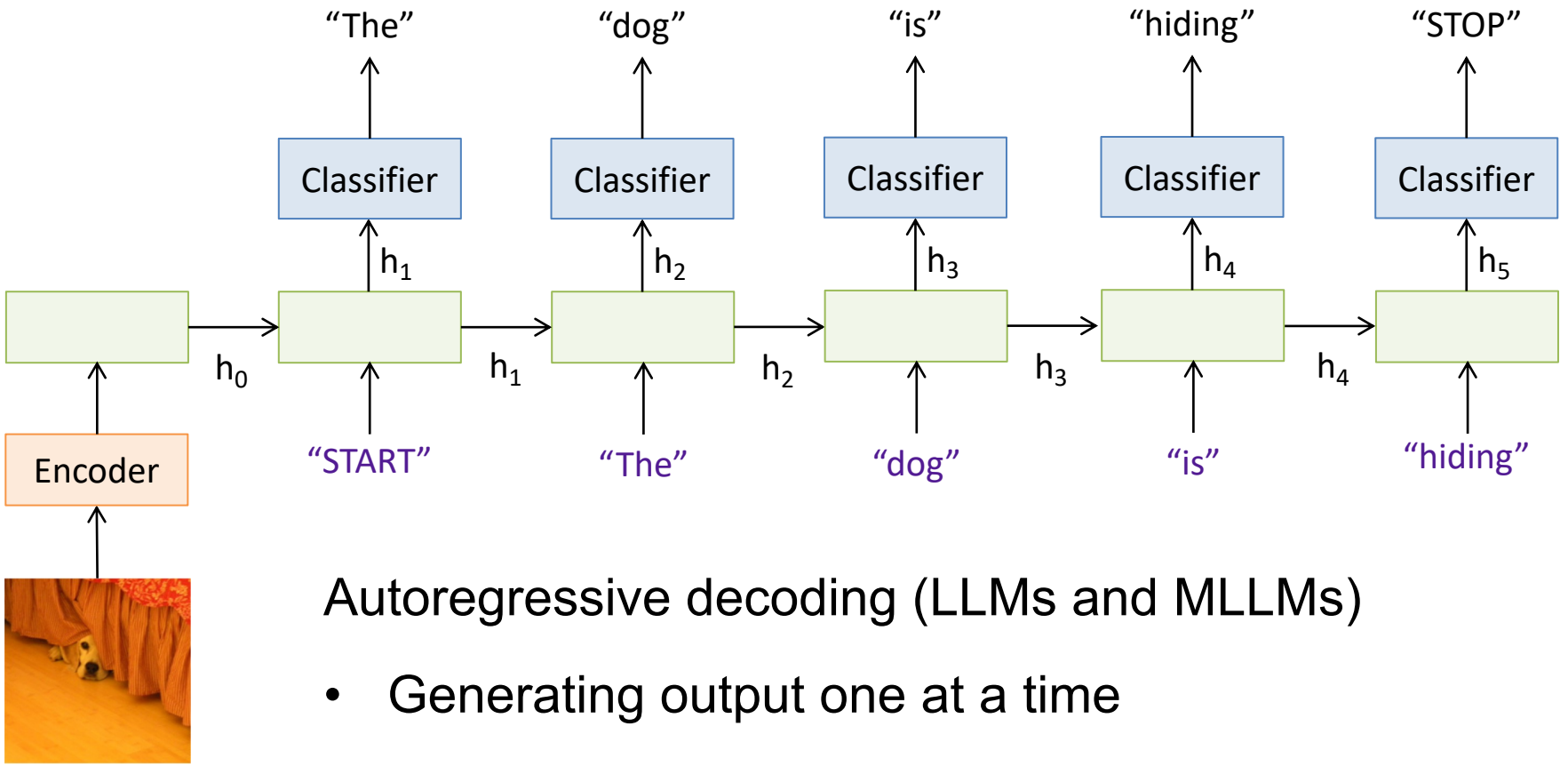
Transformers
(Part I)

Many slides from Lana Lazebnik, Arun Mallya

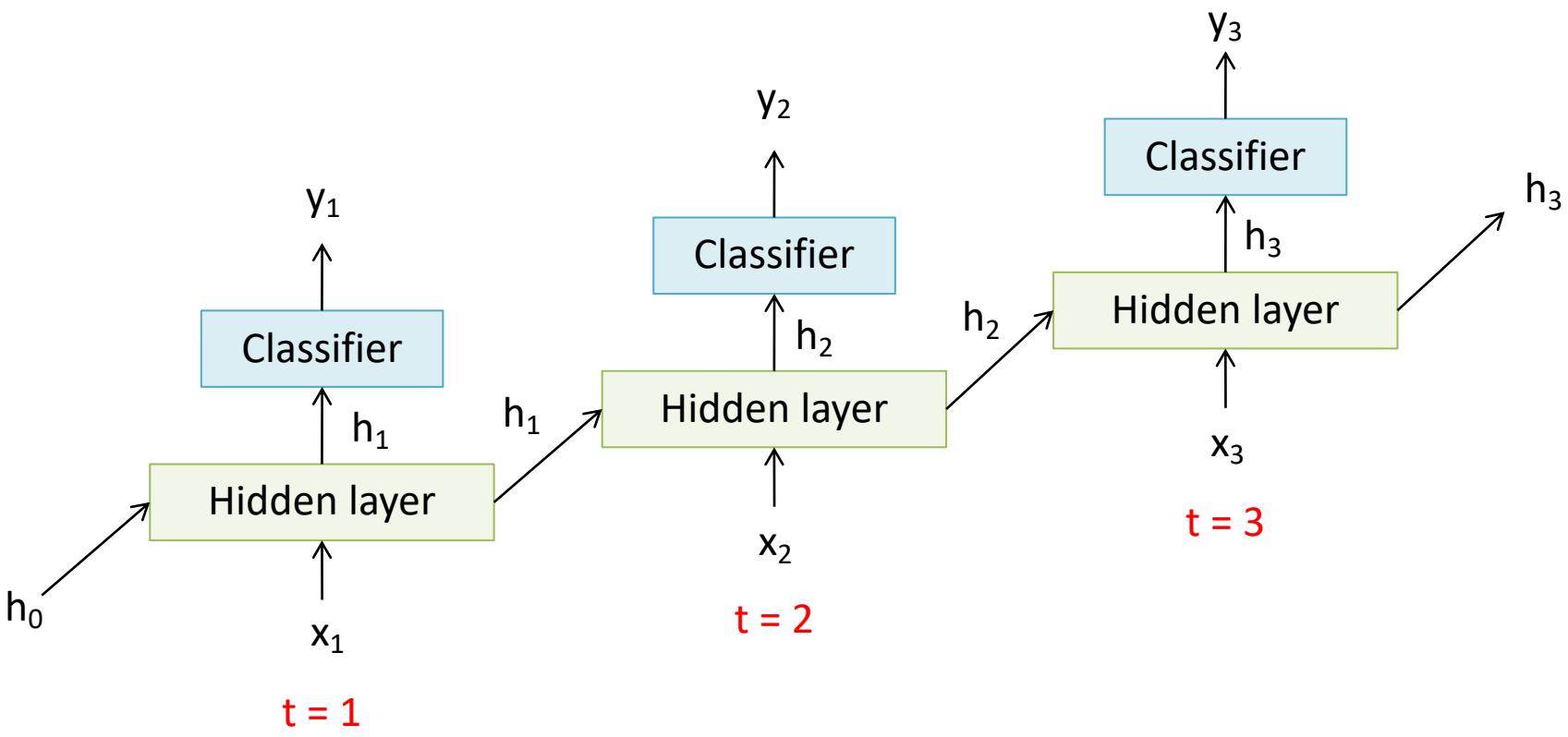
Recall: Image Caption Generation



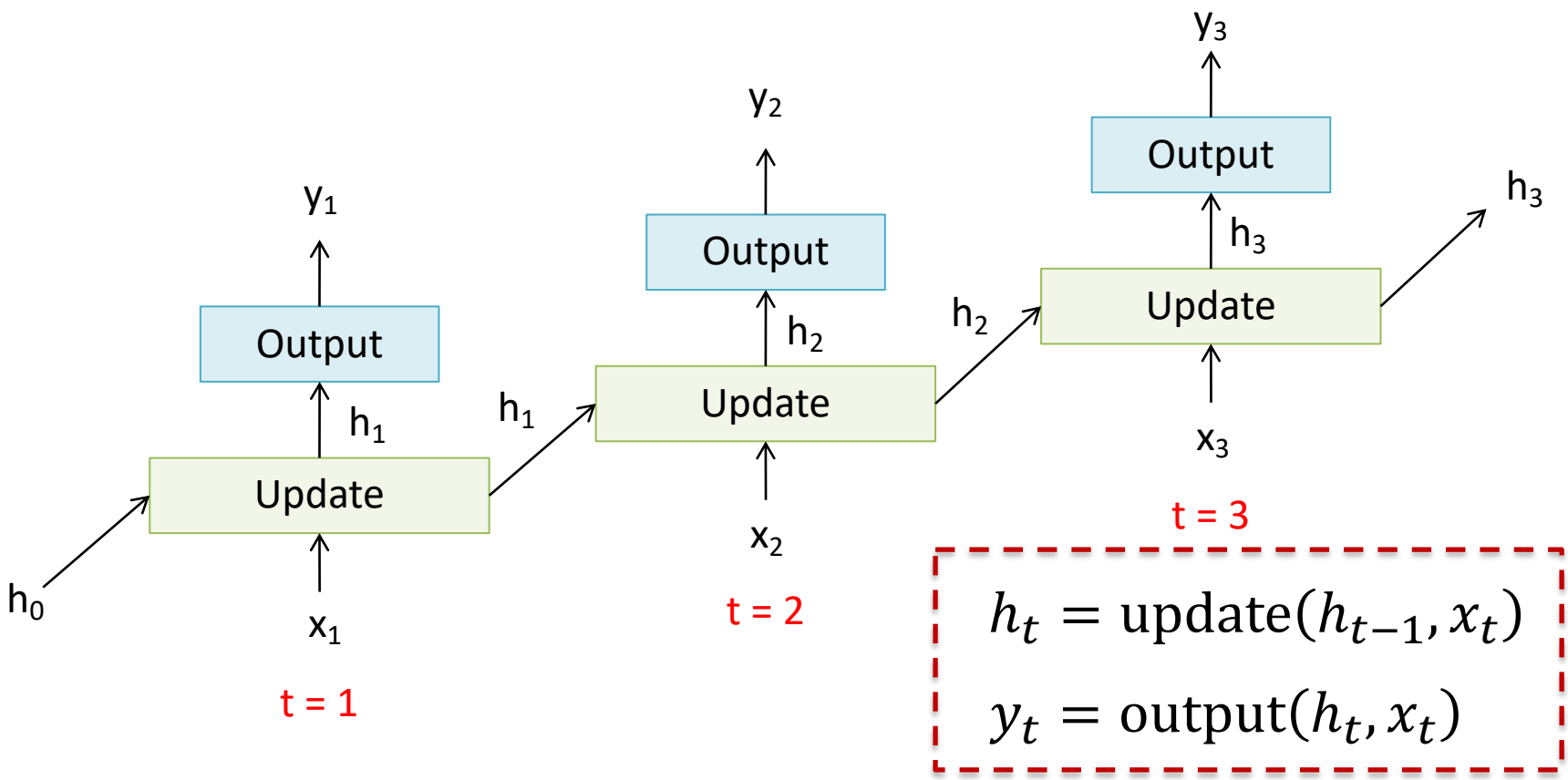
Autoregressive decoding (LLMs and MLLMs)

- Generating output one at a time
- Feeding model output at $t-1$ as the input at t

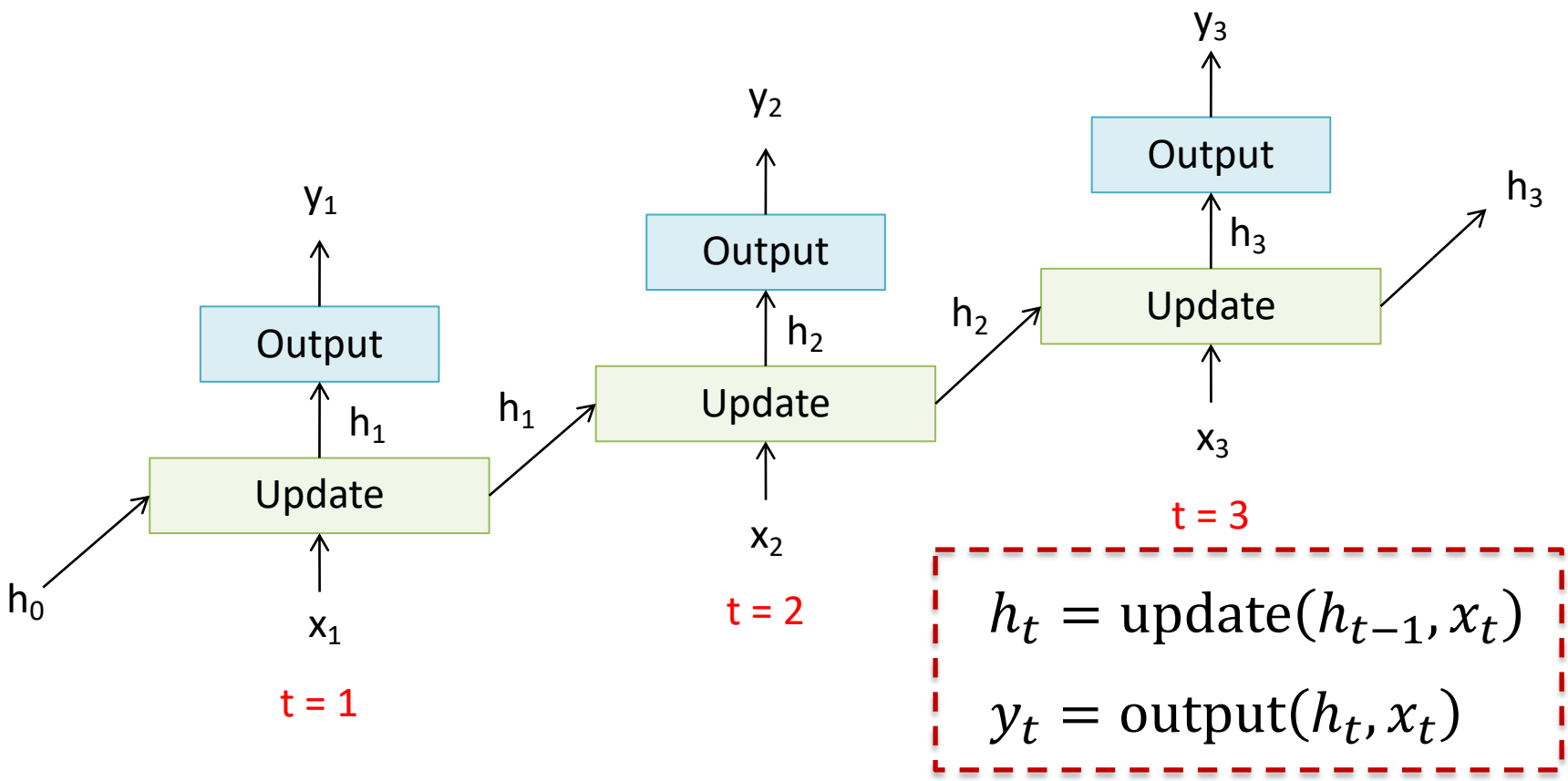
From RNNs to Transformers



From RNNs to Transformers



From RNNs to Transformers



RNNs: the state *h* is a fixed-size vector and the update function follows an RNN cell

Memory in RNNs

Recurrence:

$$h_t = f_W\,(x_t, h_{t-1})$$

current state | function of W | input at time t | old state

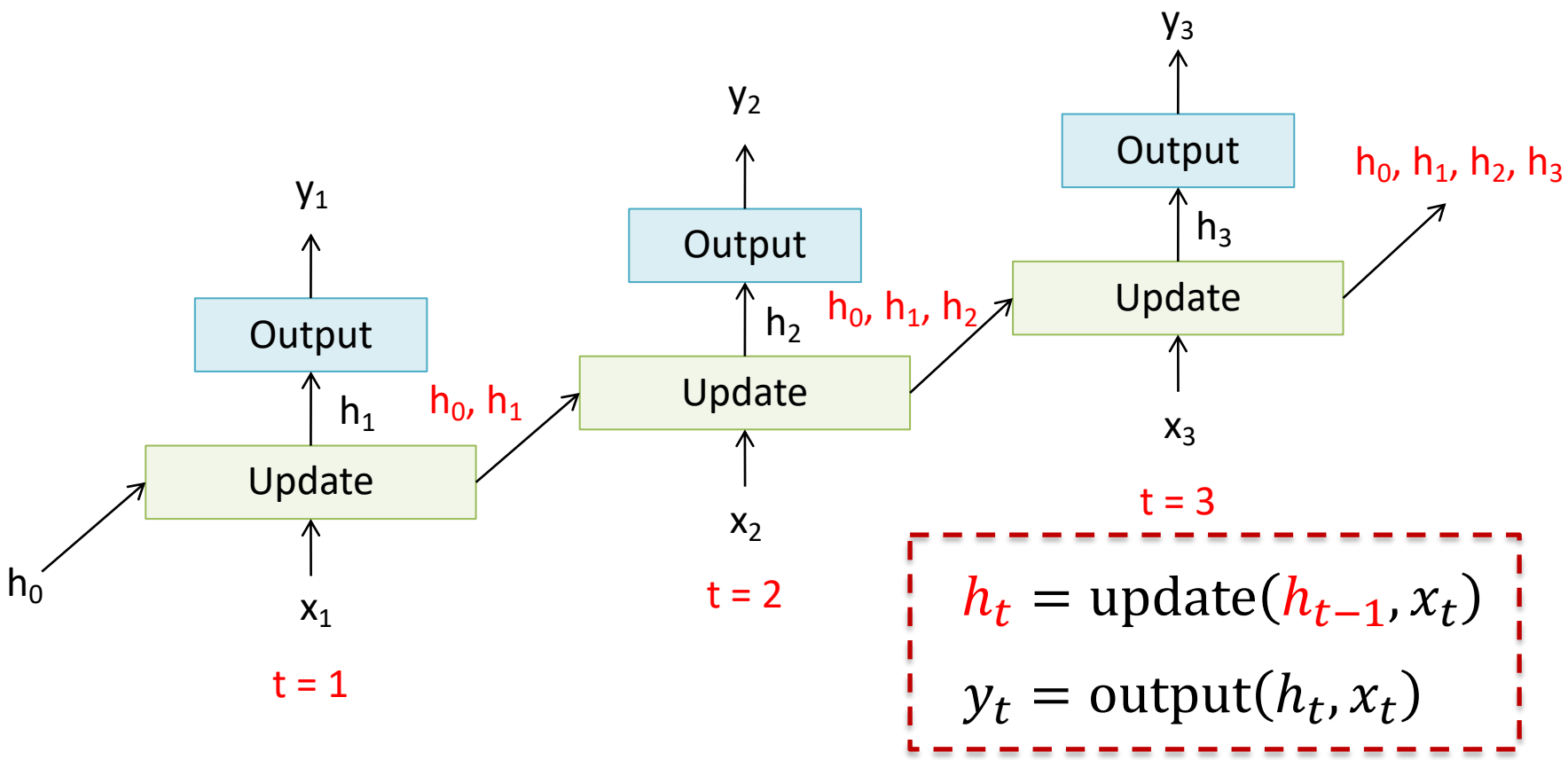
Memory:

$$h_t = \Phi\,(x_{1:t}, h_0)$$

current state | input up to time t | initial state

- RNNs “remember” an input sequence using a single vector
- What if we keep the full list of hidden states $\widehat{h_t} = [h_0, h_1, \ldots h_t]$?

From RNNs to Transformers



Now *h* is **a list of all previous hidden states**

From RNNs to Transformers

$$h_t = \text{update}(h_{t-1}, x_t)$$

$$y_t = \text{output}(h_t, x_t)$$

h is **a list of all previous states**

Update $h_t = h_{t-1}.\text{append}(f(x_t))$

Output $y_t = g(h_t, x_t)$

Transformers are one instantiation

From RNNs to Transformers

$$h_t = \text{update}(h_{t-1}, x_t)$$

$$y_t = \text{output}(h_t, x_t)$$

h is a list of
all previous states

Query $q_t = W_Q^T x_t$ **K**ey $k_t = W_K^T x_t$ **V**alue $v_t = W_V^T x_t$

Update $h_t = h_{t-1}.\text{append}([k_t, v_t])$

From RNNs to Transformers

$$h_t = \text{update}(h_{t-1}, x_t)$$

$$y_t = \text{output}(h_t, x_t)$$

h is a list of all previous states

Query $q_t = W_Q^T x_t$ **K**ey $k_t = W_K^T x_t$ **V**alue $v_t = W_V^T x_t$

Update $h_t = h_{t-1}.\text{append}([k_t, v_t])$

Output $y_t = \sum_{i=0}^{t} \text{similarity}(q_t, k_i)\, v_i$

From RNNs to Transformers

$$h_t = \text{update}(h_{t-1}, x_t)$$
$$y_t = \text{output}(h_t, x_t)$$

h is a list of
all previous states

Query $q_t = W_Q^T x_t$ **K**ey $k_t = W_K^T x_t$ **V**alue $v_t = W_V^T x_t$

Update $h_t = h_{t-1}.\text{append}([k_t, v_t])$

Output $y_t = \sum_{i=0}^{t} \text{similarity}(q_t, k_i)\, v_i$ Key retrieval

From RNNs to Transformers

$$h_t = \text{update}(h_{t-1}, x_t)$$
$$y_t = \text{output}(h_t, x_t)$$

h is a list of all previous states

Query $q_t = W_Q^T x_t$ **K**ey $k_t = W_K^T x_t$ **V**alue $v_t = W_V^T x_t$

Update $h_t = h_{t-1}.\text{append}([k_t, v_t])$

Output $y_t = \sum_{i=0}^{t} \text{similarity}(q_t, k_i)\, v_i$

Weighted sum of values

From RNNs to Transformers

$$h_t = \text{update}(h_{t-1}, x_t)$$

$$y_t = \text{output}(h_t, x_t)$$

h is a list of all previous states

Query $q_t = W_Q^T x_t$ **K**ey $k_t = W_K^T x_t$ **V**alue $v_t = W_V^T x_t$

Update $h_t = h_{t-1}.\text{append}([k_t, v_t])$

Output $y_t = \sum_{i=0}^{t} \text{similarity}(q_t, k_i)\, v_i$

$V_t = [v_1, \ldots, v_t]^T \in R^{t \times d}$ $= \text{softmax}([q_t^T k_1, q_t^T k_2, \ldots, q_t^T k_t])V_t$

From RNNs to Transformers

$$h_t = \text{update}(h_{t-1}, x_t)$$
$$y_t = \text{output}(h_t, x_t)$$

h is a list of all previous states

Query $q_t = W_Q^T x_t$ **K**ey $k_t = W_K^T x_t$ **V**alue $v_t = W_V^T x_t$

Update $h_t = h_{t-1}.\text{append}([k_t, v_t])$

Output $y_t = \sum_{i=0}^{t} \text{similarity}(q_t, k_i)\, v_i$

$V_t = [v_1, \dots, v_t]^T \in R^{t \times d}$ $= \underline{\text{softmax}}([\underline{q_t^T k_1, q_t^T k_2, \dots, q_t^T k_t}])V_t$

Normalization Inner product as similarity

From RNNs to Transformers

$$h_t = \text{update}(h_{t-1}, x_t)$$
$$y_t = \text{output}(h_t, x_t)$$

h is a list of all previous states

Query $q_t = W_Q^T x_t$ **K**ey $k_t = W_K^T x_t$ **V**alue $v_t = W_V^T x_t$

Update $h_t = h_{t-1}.\text{append}([k_t, v_t])$

Output $y_t = \sum_{i=0}^{t} \text{similarity}(q_t, k_i)\, v_i$

$V_t = [v_1, \dots, v_t]^T \in R^{t \times d}$ $= \text{softmax}([q_t^T k_1, q_t^T k_2, \dots, q_t^T k_t])V_t$

$K_t = [k_1, \dots, k_t]^T \in R^{t \times d}$ $= \text{softmax}([q_t^T K_t^T])V_t$

From RNNs to Transformers

$$h_t = \text{update}(h_{t-1}, x_t)$$
$$y_t = \text{output}(h_t, x_t)$$

h is a list of all previous states

Query $q_t = W_Q^T x_t$ **K**ey $k_t = W_K^T x_t$ **V**alue $v_t = W_V^T x_t$

Update $h_t = h_{t-1}.\text{append}([k_t, v_t])$

Output $y_t = \sum_{i=0}^{t} \text{similarity}(q_t, k_i)\, v_i$

$V_t = [v_1, \dots, v_t]^T \in R^{t \times d}$ $= \text{softmax}([q_t^T k_1, q_t^T k_2, \dots, q_t^T k_t])V_t$

$K_t = [k_1, \dots, k_t]^T \in R^{t \times d}$ $= \text{softmax}([q_t^T K_t^T])V_t$ Self-Attention!

From RNNs to Transformers

$$h_t = \text{update}(h_{t-1}, x_t)$$

$$y_t = \text{output}(h_t, x_t)$$

RNN

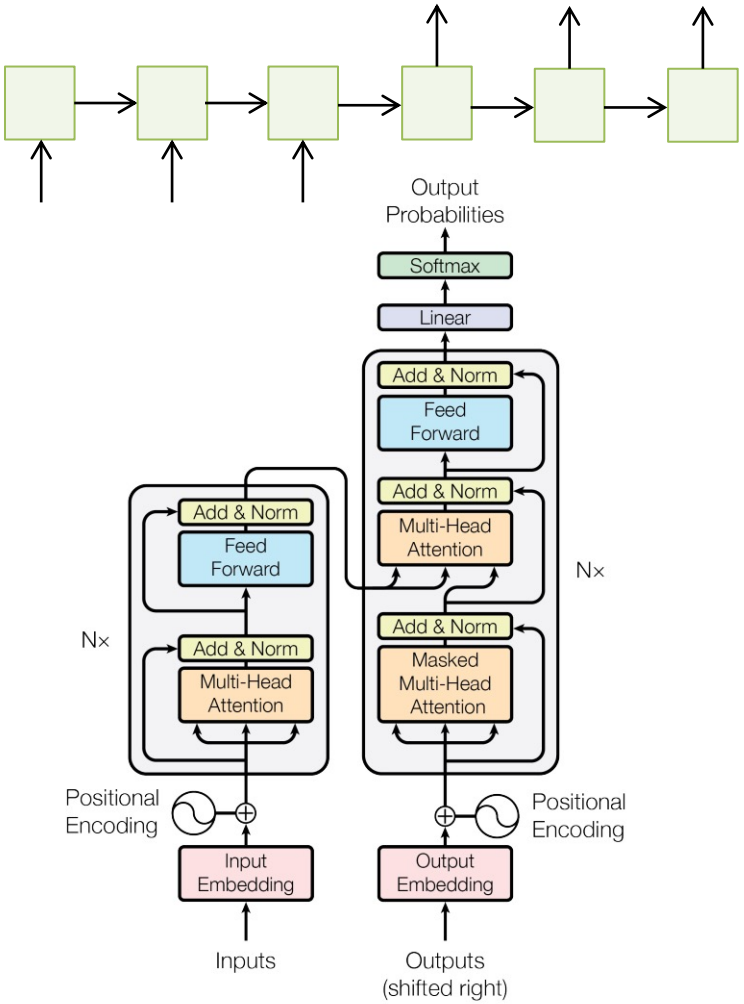
- State *h* is a vector
- Update using RNN cell
- Output using MLP

Self-Attention (Transformer)

- State *h* is a list
- Update by list appending
- Output using re-weighting (weights from softmax)

Transformer: An Overview

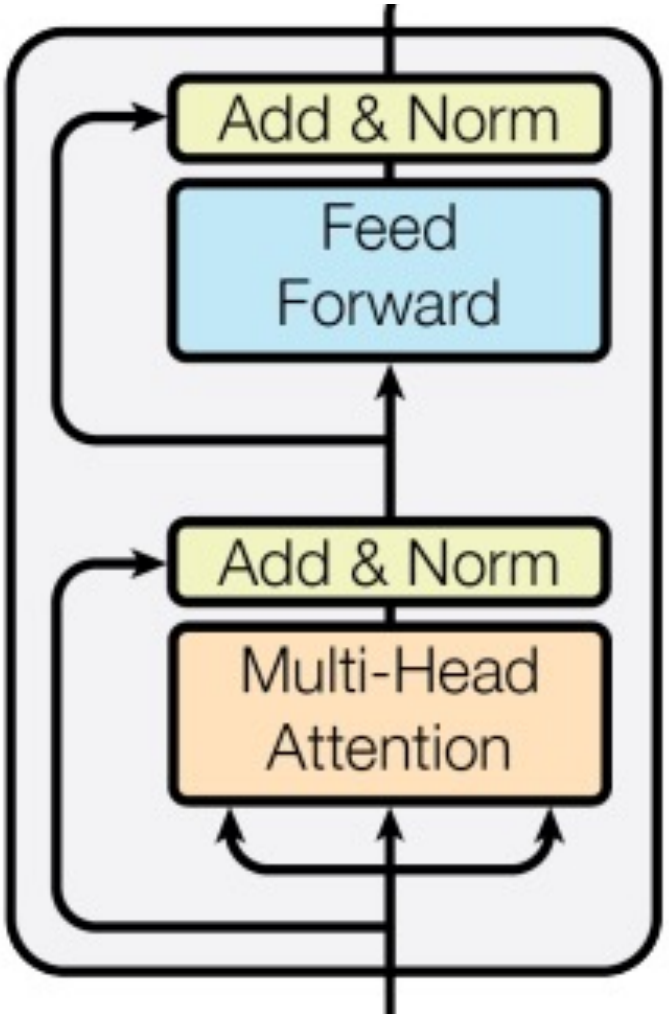
- Initially developed for **sequence modeling** in NLP
- Later adopted as **feature transforms** in vision
- Replace RNNs with self-attention + MLP
- Enormously successful in both NLP and vision



Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., ... & Polosukhin, I. (2017). Attention is all you need. In *NeurIPS* (pp. 5998-6008).

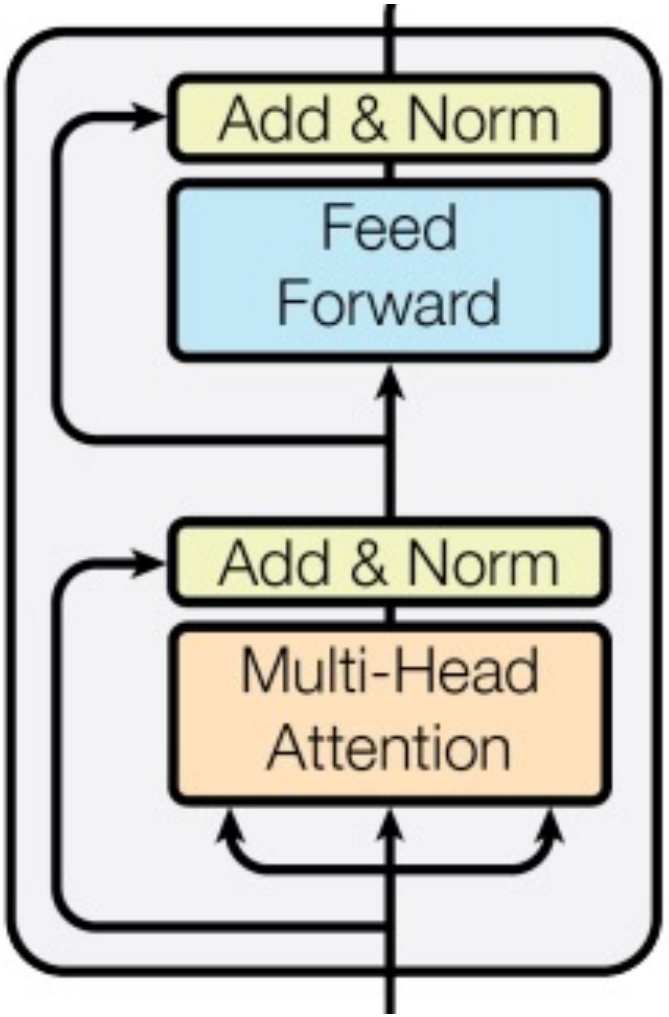
Transformer: The Basic Block

- Multi-head self-attention
- Normalization
- Feedforward network (MLP)



Transformer: The Basic Block

- Multi-head self-attention
- Normalization
- Feedforward network (MLP)

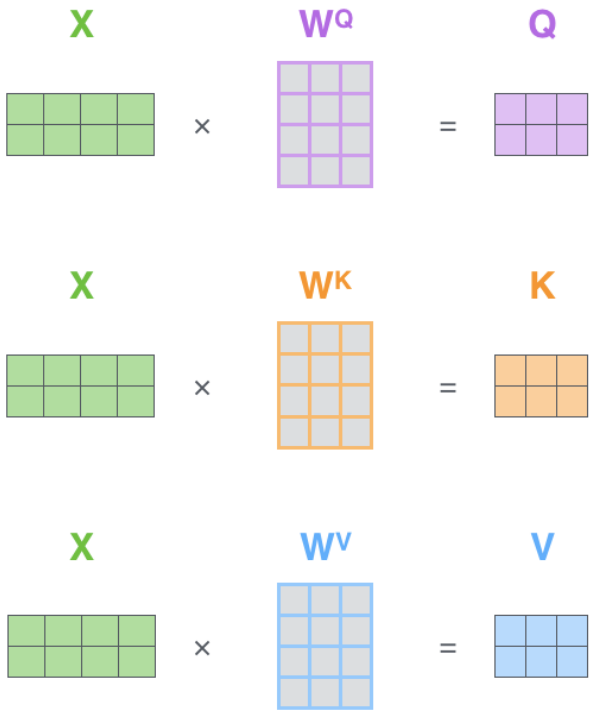


Self-Attention as Feature Transform

- Input: a 1D feature map $X \in R^{T \times d}$
- Output: transformed 1D feature map $Z \in R^{T \times d_k}$

Self-Attention as Feature Transform

- Input: a 1D feature map $X \in R^{T \times d}$
- Step 1: project X into query (Q), key (K), and value (V)

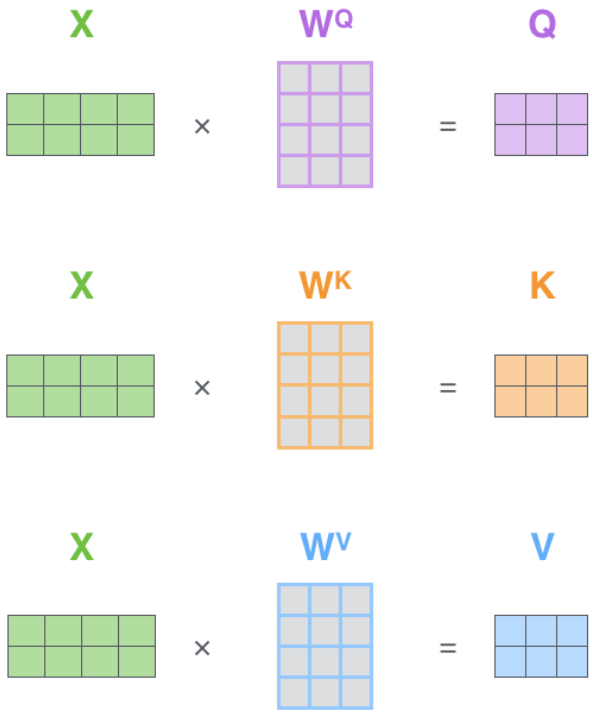


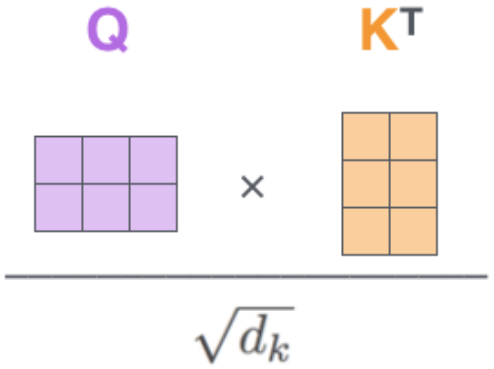
Size: $T \times d_k$

Slide Credit: Jay Alammar. The Illustrated Transformer

Self-Attention as Feature Transform

- Input: a 1D feature map $X \in R^{T \times d}$
- Step 2: compute the product between queries & keys



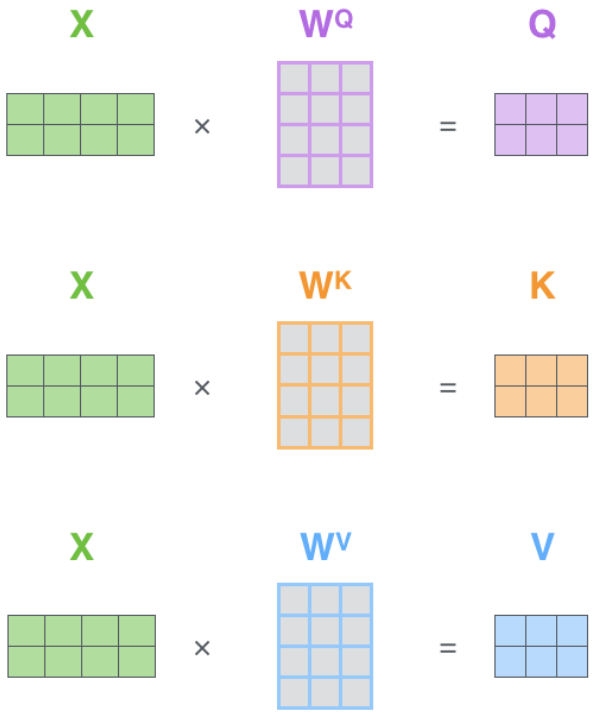


Size: $T \times T$

Slide Credit: Jay Alammar. The Illustrated Transformer

Self-Attention as Feature Transform

- Input: a 1D feature map $X \in R^{T \times d}$
- Step 3: normalize the product into similarity scores



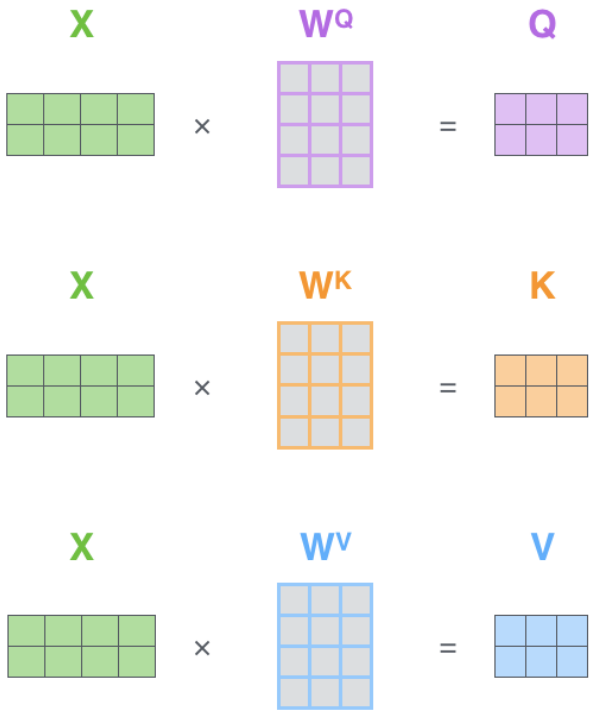
$$\text{softmax}\left(\frac{Q \times K^T}{\sqrt{d_k}}\right)$$

Each Row sum to 1. Size: $T \times T$

Slide Credit: Jay Alammar. The Illustrated Transformer

Self-Attention as Feature Transform

- Input: a 1D feature map $X \in R^{T \times d}$
- Step 4: multiple the similarity scores with values

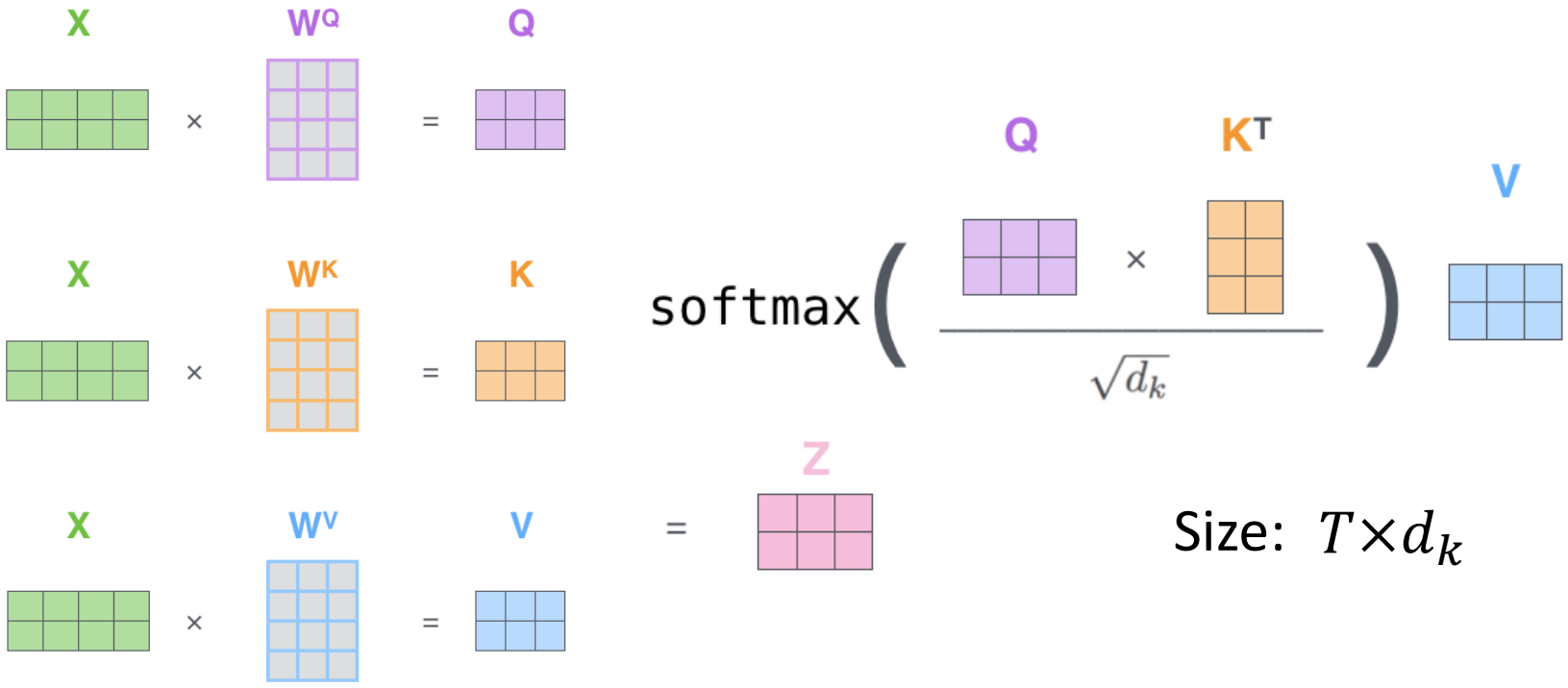


$$\text{softmax}\left(\frac{Q \times K^T}{\sqrt{d_k}}\right) V$$

Slide Credit: Jay Alammar. The Illustrated Transformer

Self-Attention as Feature Transform

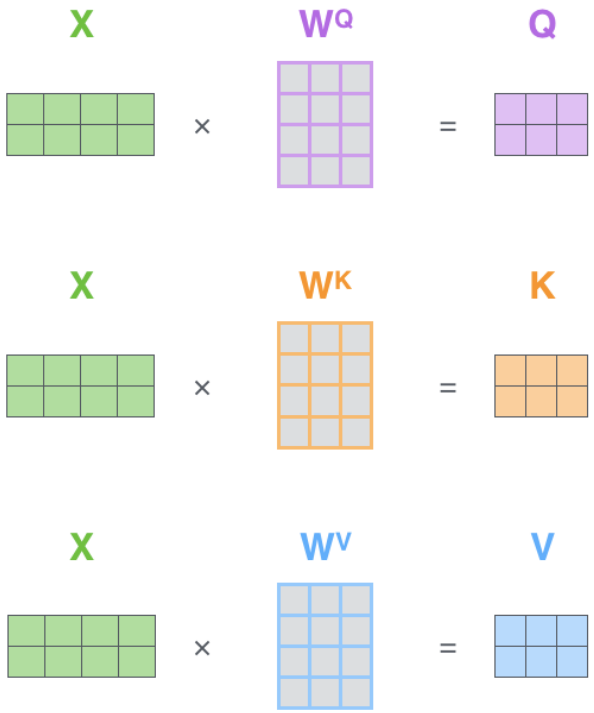
- Input: a 1D feature map $X \in R^{T \times d}$
- Output: transformed 1D feature map $z \in R^{T \times d_k}$



Slide Credit: Jay Alammar. The Illustrated Transformer

Self-Attention as Feature Transform

- Input: a 1D feature map $X \in R^{T \times d}$
- Output: transformed 1D feature map $z \in R^{T \times d_k}$



self-similarity (bilinear w.r.t. X)

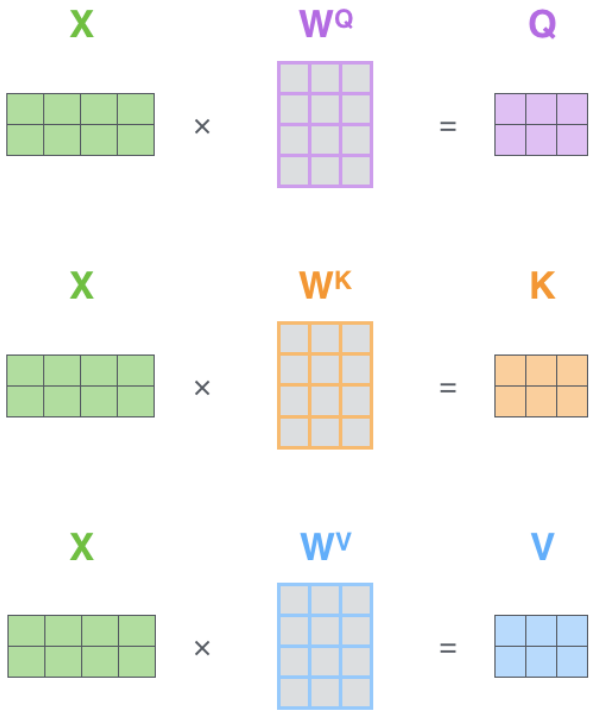
$$Softmax\left(\frac{XW^Q(W^K)^T X^T}{\sqrt{d_k}}\right) XW^V$$

$$= Z$$

Slide Credit: Jay Alammar. The Illustrated Transformer

Self-Attention as Feature Transform

- Input: a 1D feature map $X \in R^{T \times d}$
- Output: transformed 1D feature map $z \in R^{T \times d_k}$



Channel-wise, data-dependent kernel

$$Softmax\left(\frac{XW^Q(W^K)^T X^T}{\sqrt{d_k}}\right) XW^V$$

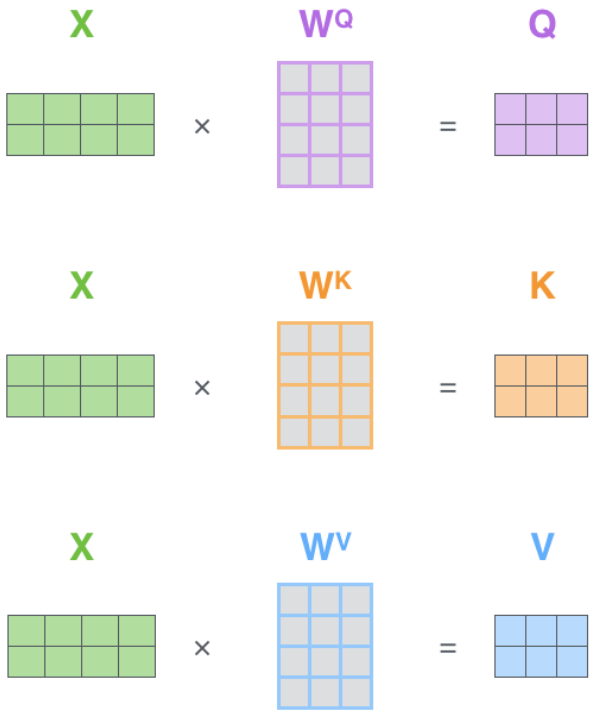
$$= Z$$

Each row of the self-attention matrix forms a kernel applied to one feature channel across all time steps

Slide Credit: Jay Alammar. The Illustrated Transformer

Self-Attention as Feature Transform

- Input: a 1D feature map $X \in R^{T \times d}$
- Output: transformed 1D feature map $z \in R^{T \times d_k}$



Channel-wise, data-dependent kernel

$$Softmax\left(\frac{XW^Q(W^K)^T X^T}{\sqrt{d_k}}\right) XW^V$$

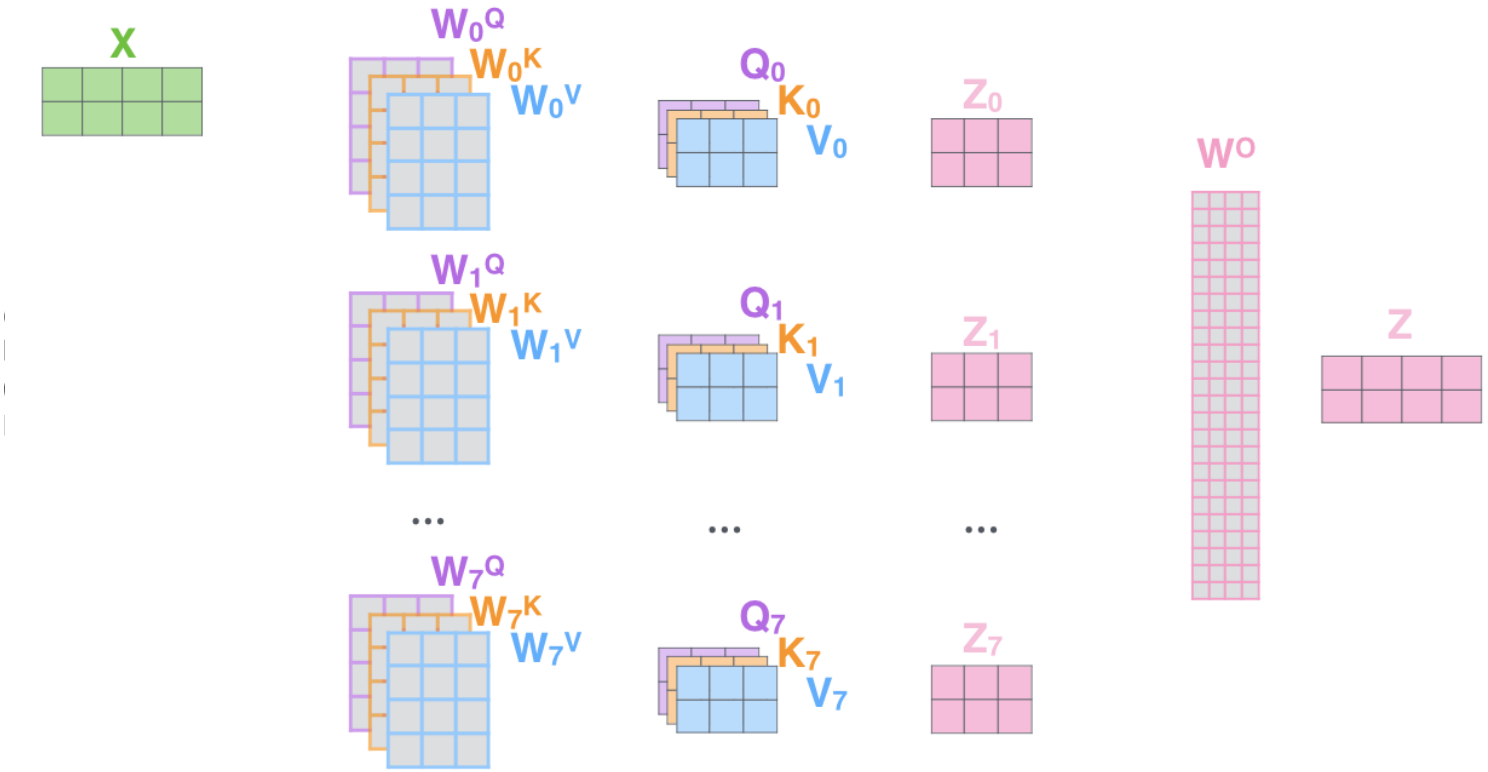
$$= Z$$

At least as expressive as convolutions
(See Cordonnier et al. ICLR 2020)

Slide Credit: Jay Alammar. The Illustrated Transformer

Multi-head Self-Attention

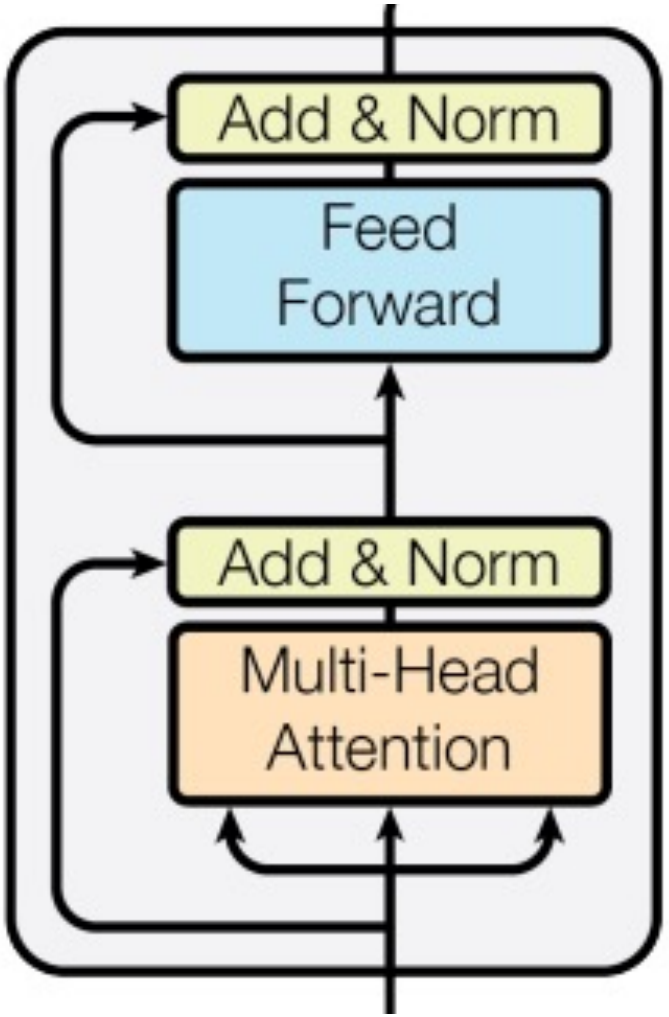
- Compute each head independently
- Concatenate the results & re-project using a linear layer



Slide Credit: Jay Alammar. [The Illustrated Transformer]

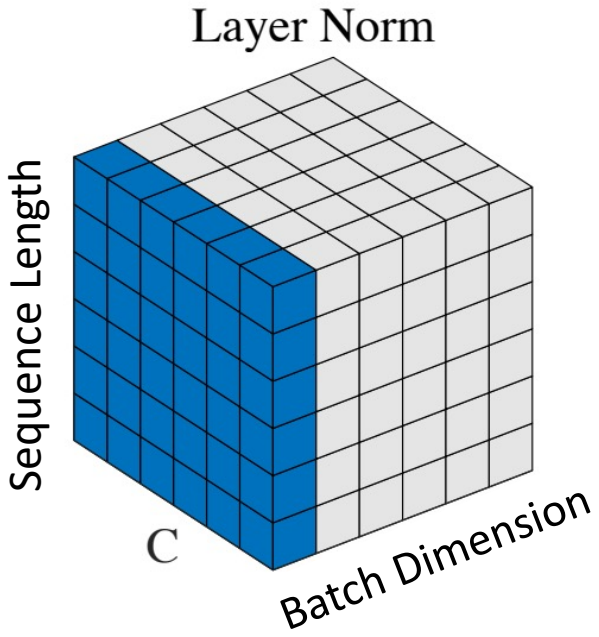
Transformer: The Basic Block

- Multi-head self-attention
- Normalization
- Feedforward network (MLP)



Layer Norm

- Input: a 1D feature map *Z*
- Output: transformed 1D feature map *Z'*

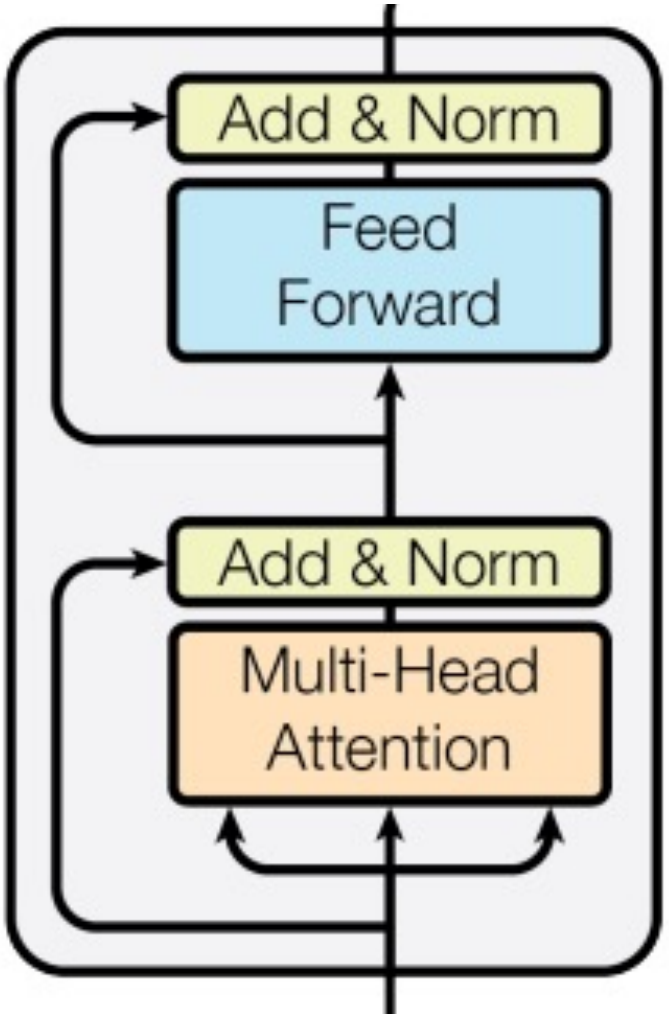


$$z'_{(t)} = \frac{z_{(t)} - E[z_{(t)}]}{\sqrt{Var[z_{(t)}]}}$$

Normalization within each sample
Can easily handle "empty" spots

Transformer: The Basic Block

- Multi-head self-attention
- Normalization
- Feedforward network (MLP)



Feedforward Network (FFN)

- Input: a 1D feature map *Z'*
- Output: transformed 1D feature map *R*
- Simply a two-layer MLP (with GELU as the non-linear activation function)

Hendrycks, D., & Gimpel, K. (2016). Gaussian error linear units (gelus). *arXiv preprint arXiv:1606.08415*.

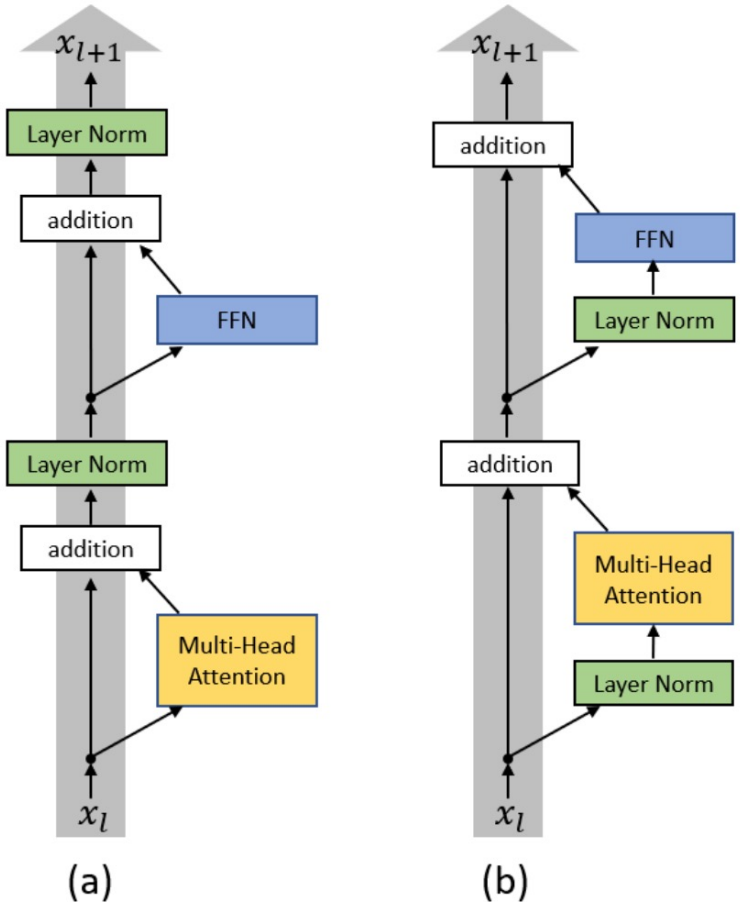
Feedforward Network (FFN)

- Input: a 1D feature map *Z'*
- Output: transformed 1D feature map *R*
- Simply a two-layer MLP (with GELU as the non-linear activation function)
- Inverted bottleneck: dimension expansion followed by reduction (as additional memory?)

Geva, M., Schuster, R., Berant, J., & Levy, O. (2021). Transformer Feed-Forward Layers Are Key-Value Memories. *EMNLP*

Transformer: The Basic Block

- Post-LN vs Pre-LN
- Difficulty in training. Warm-up plus adaptive gradients are often needed.
- Reference Implementation (~60 lines)



Xiong, R., Yang, Y., He, D., Zheng, K., Zheng, S., Xing, C., ... & Liu, T. (2020). On layer normalization in the transformer architecture. In *ICML* (pp. 10524-10533). PMLR.

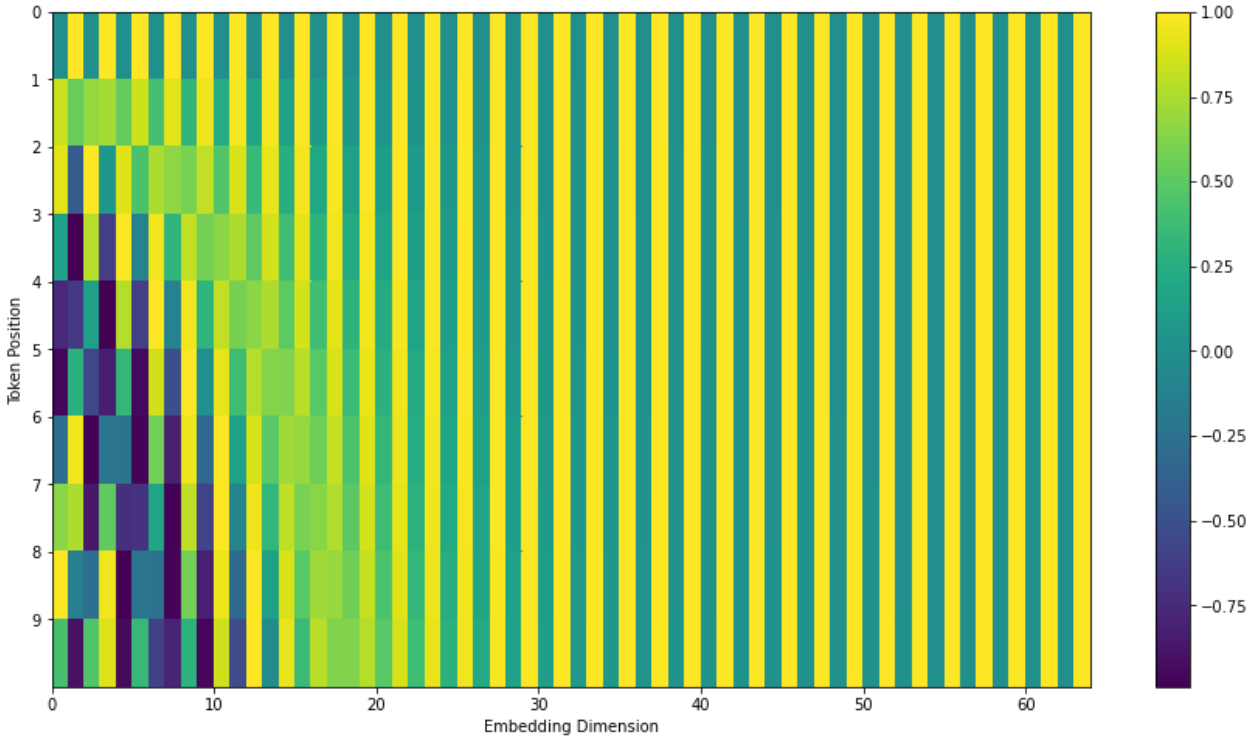
Transformer: Issues

- Computational and memory intensive
  - Need O(T^2) (T as sequence length) to compute & store the self-attention
  - Linear self-attention (more later …)
  - Efficient approximation of self-attention (e.g., constraining the computation of self-attention in a neighborhood)

- Permutation invariant, i.e., permuting the input = permuting the output
  - Adding position encoding to inputs

Position Encoding

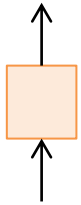
- A lookup table (either fixed or learnable)
- Map a time step to a high dimensional vector



Slide Credit: Jay Alammar. [The Illustrated Transformer]

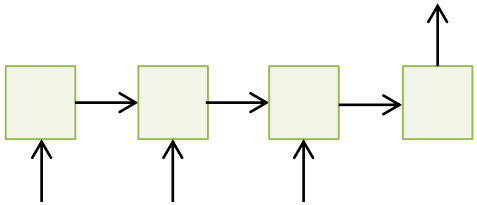
Transformers: Use Cases

Single - Single



Feed-forward Network
(as feature transform)

Multiple - Single



Sentiment Classification
(as sequence modeling)

Single - Multiple

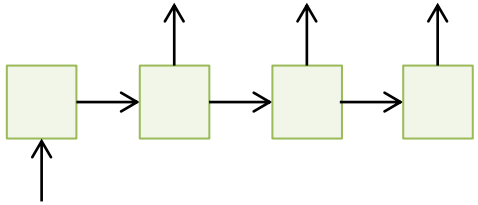
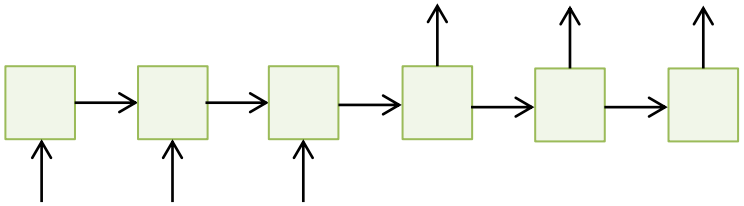


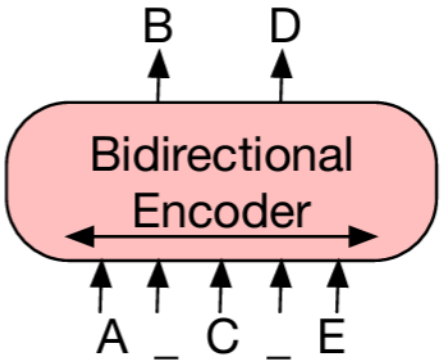
Image Captioning
(as sequence modeling)

Multiple - Multiple

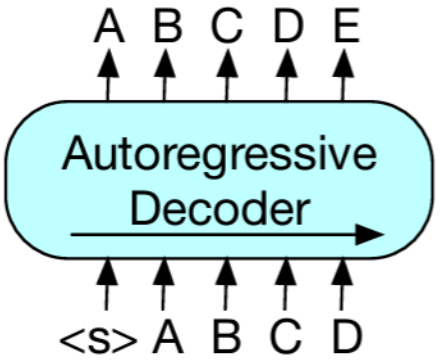


Translation / Chatbot
(as sequence modeling)

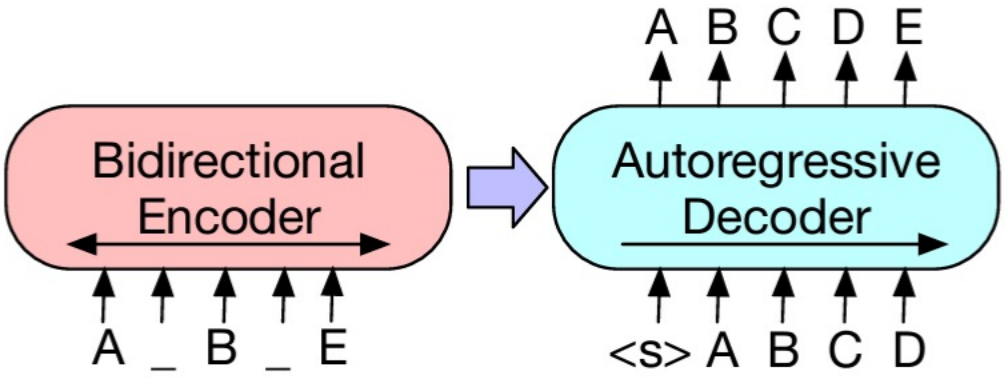
Transformers for Sequence Modeling



Encoder-only architecture



Decoder-only architecture



Encoder-decoder architecture

Figure source: Lewis et al., 2020

Transformers: Summary

- A Transformer block can be used as *feature transform or as sequence modeling*
- A Transformer network is built on Transformer blocks
- Capable of capturing long-range dependencies within an input sequence
- Transformer variants are now behind many AI solutions